

""""

Лабораторна робота №8

Варіант: $F(x) = \sin(x) - x/2 = 0$, відрізок $[-6, 6]$

Алгебраїчне рівняння: $x^3 - x^2 + 4x - 4 = 0$

Точність: $\text{eps} = 1e-10$

""""

```
import math
```

```
import cmath
```

```
EPS = 1e-10
```

```
MAX_ITER = 1000
```

```
POLY_COEFFS = [1, -1, 4, -4]
```

```
# — Трансцендентна функція та похідні —
```

```
def f_trans(x): return math.sin(x) - x / 2.0
```

```
def df_trans(x): return math.cos(x) - 0.5
```

```
def d2f_trans(x): return -math.sin(x)
```

```
def stop_criteria(x_new, x_old, eps=EPS):
```

```
    return abs(x_new - x_old) < eps and abs(f_trans(x_new)) < eps
```

```
# — 1. Ітераційні методи —
```

```
def simple_iteration(x0):
```

```
    deriv0 = df_trans(x0)
```

```
    weight = 1.0 / deriv0 if abs(deriv0) > 1e-12 else 1.0
```

```
    xk = x0
```

```
    for i in range(1, MAX_ITER + 1):
```

```
        x_next = xk - weight * f_trans(xk)
```

```
        if stop_criteria(x_next, xk): return x_next, i
```

```
    xk = x_next  
    return xk, MAX_ITER
```

```
def newton(x0):
```

```
    xk = x0  
  
    for i in range(1, MAX_ITER + 1):  
        x_next = xk - f_trans(xk) / df_trans(xk)  
        if stop_criteria(x_next, xk): return x_next, i  
        xk = x_next  
    return xk, MAX_ITER
```

```
def chebyshev(x0):
```

```
    xk = x0  
  
    for i in range(1, MAX_ITER + 1):  
        fv, dfv, d2fv = f_trans(xk), df_trans(xk), d2f_trans(xk)  
        x_next = xk - fv / dfv - (d2fv * fv**2) / (2.0 * dfv**3)  
        if stop_criteria(x_next, xk): return x_next, i  
        xk = x_next  
    return xk, MAX_ITER
```

```
def chord_method(x0):
```

```
    x_prev, x_cur = x0, x0 + 0.1  
  
    for i in range(1, MAX_ITER + 1):  
        diff_f = f_trans(x_cur) - f_trans(x_prev)  
        x_next = x_cur - f_trans(x_cur) * (x_cur - x_prev) / diff_f  
        if stop_criteria(x_next, x_cur): return x_next, i  
        x_prev, x_cur = x_cur, x_next  
    return x_cur, MAX_ITER
```

```
def parabola_method(x0):
```

```

d = 0.1
pts = [x0, x0 + d, x0 + 2*d]
for i in range(1, MAX_ITER + 1):
    f0, f1, f2 = f_trans(pts[0]), f_trans(pts[1]), f_trans(pts[2])
    h1, h2 = pts[1] - pts[0], pts[2] - pts[1]
    d1, d2 = (f1 - f0)/h1, (f2 - f1)/h2
    dd = (d2 - d1)/(h1 + h2)
    bb = d2 + h2 * dd
    disc = bb**2 - 4.0 * f2 * dd
    sqrt_disc = math.sqrt(disc) if disc > 0 else 0.0
    den = bb + sqrt_disc if abs(bb + sqrt_disc) > abs(bb - sqrt_disc) else bb - sqrt_disc
    x_next = pts[2] - 2.0 * f2 / den
    if stop_criteria(x_next, pts[2]): return x_next, i
    pts = [pts[1], pts[2], x_next]
return pts[2], MAX_ITER

```

— 2. Алгебраїчні рівняння (Горнер та Лін) —

```

def horner_eval(coeffs, arg):
    b = [0.0] * len(coeffs)
    b[0] = coeffs[0]
    for i in range(1, len(coeffs)): b[i] = b[i-1] * arg + coeffs[i]
    c = [0.0] * len(coeffs)
    c[0] = b[0]
    for i in range(1, len(coeffs) - 1): c[i] = c[i-1] * arg + b[i]
    return b[-1], c[-2], b[:-1]

```

```

def newton_horner(coeffs, x0):
    xk = x0
    for i in range(1, MAX_ITER + 1):
        pval, dpval, _ = horner_eval(coeffs, xk)

```

```

x_next = xk - pval / dpval
if abs(x_next - xk) < EPS and abs(pval) < EPS: return x_next, i
xk = x_next
return xk, MAX_ITER

```

```

def lin_method(coeffs, p0, q0):

```

```

    a = [float(c) for c in coeffs]
    pp, qq = float(p0), float(q0)
    for i in range(1, MAX_ITER + 1):
        b0 = a[0]
        b1 = a[1] - pp * b0
        r1 = a[2] - pp * b1 - qq * b0
        r0 = a[3] - qq * b1
        j11, j12, j21, j22 = -b1 + pp*b0, -b0, qq*b0, -b1
        det = j11 * j22 - j12 * j21
        dp = (-r1 * j22 + r0 * j12) / det
        dq = (r1 * j21 - r0 * j11) / det
        pp += dp
        qq += dq
        if abs(r1) < EPS and abs(r0) < EPS:
            disc = complex(pp**2 - 4.0 * qq)
            return (-pp + cmath.sqrt(disc)) / 2.0, (-pp - cmath.sqrt(disc)) / 2.0, i
    return 0, 0, MAX_ITER

```

```

def main():

```

```

    print("— ЧАСТИНА 1. Трансцендентне рівняння (eps = 1e-10) —")
    root_rising, root_falling = 0.0, 1.8954
    methods = [("Проста ітерація", simple_iteration), ("Ньютона", newton),
               ("Чебишева", chebyshev), ("Хорд", chord_method), ("Парабол", parabola_method)]

```

```
for label, start in [("f">0 (зростання)", root_rising), ("f"<0 (спадання)", root_falling)]:
```

```
    print(f"\nКорень {label}, Початкове x0 = {start}:")
```

```
    for name, method in methods:
```

```
        root, iters = method(start)
```

```
        print(f" {name:<18} | x* = {root:14.10f} | Ітерацій: {iters}")
```

```
print("\n— ЧАСТИНА 2. Алгебраїчне рівняння  $x^3 - x^2 + 4x - 4 = 0$  —")
```

```
r_root, r_iters = newton_horner(POLY_COEFFS, 0.0)
```

```
print(f"Дійсний корінь (Ньютон-Горнер): x = {r_root:.10f} (Ітерацій: {r_iters})")
```

```
c_root1, c_root2, c_iters = lin_method(POLY_COEFFS, 0.5, 3.5)
```

```
print(f"Комплексні корені (Метод Ліна): x1 = {c_root1.real:.0f}+{c_root1.imag:.0f}j, "
```

```
    f"x2 = {c_root2.real:.0f}{c_root2.imag:.0f}j (Ітерацій: {c_iters})")
```

```
if __name__ == "__main__":
```

```
    main()
```